

Introduction to Concurrent Programming in Embedded Systems

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

Departamento de Engenharia Informática

Concurrent Programming in Embedded Systems

Classical Sequential Programming

- Traditional computation model: synchronous and linear.
- Structure: **Input** → **Processing** → **Output**.
- The program follows a terminating pattern: once the result is computed, the process ends.
- The primary focus is on the correctness of the final output.

Classical Sequential Programming

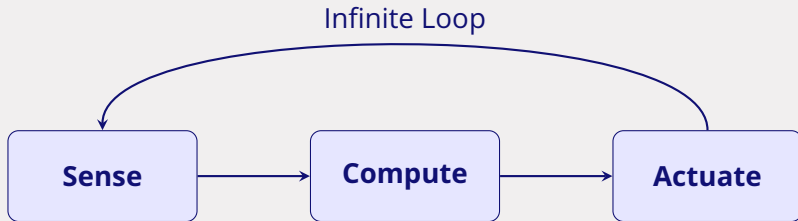
Example: A program that sums two numbers

```
int main() {  
    int a, b, sum;  
  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
  
    sum = a + b;  
  
    printf("The sum is: %d\n", sum);  
  
    return 0;  
}
```

The Embedded Systems Paradigm

- Embedded systems control physical processes.
- Embedded systems are reactive: they must respond to stimuli from the environment.
- Functions in embedded systems do not terminate; they repeat indefinitely to continuously monitor and control the environment.
- Structure: **Infinite Loop { Sense → Compute → Actuate } .**

The Embedded Systems Paradigm



The Embedded Systems Paradigm

Example: Digital weighing scale

```
while(1) {  
    weight = read_weight_sensor(); // Sense  
    display_weight(weight);        // Actuate  
}
```



Resource Management: Energy Efficiency

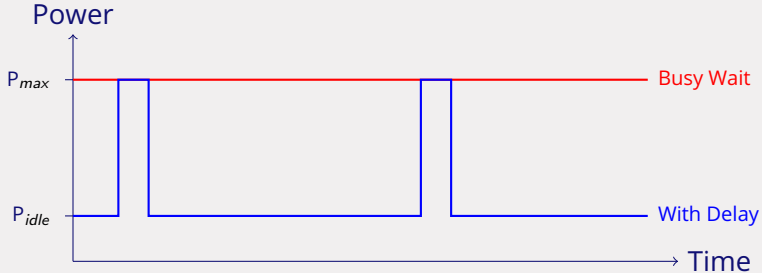
- The previous model relies on *busy waiting*
 - ▶ The loop executes incessantly, wasting CPU cycles and consuming unnecessary power.
- Physical reality often requires less frequent interaction
 - ▶ Sampling a weight every 500 ms is often sufficient to maintain functionality.
- By introducing delays, the processor can enter low-power modes during the idle intervals, significantly reducing energy consumption, while maintaining system performance.

Resource Management: Energy

Example: Digital weighing scale with energy efficiency

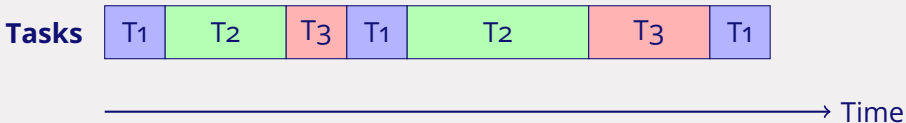
```
while(1) {  
    weight = read_weight_sensor(); // Sense  
    display_weight(weight);        // Actuate  
    delay(500);                    // Wait for 500 ms before next reading  
}
```

Resource Management: Energy Efficiency



Resource Sharing and Concurrency

- If the loop does not utilise 100% of the CPU time, the processor remains idle.
- To maximise resource utilisation, we can run multiple **tasks** concurrently on the same CPU.
- The computing platform becomes a collection of resources shared among tasks.
- **Concurrency:** Multiple tasks competing for CPU time.
- Tasks are interleaved to create the illusion of parallelism.



Concurrent Programming

- The CPU is a shared resource; tasks are interleaved to optimise its utilisation.
- **Concurrency** implies that multiple tasks progress within the same timeframe, even if they are not executing simultaneously on the processor.
- The system must manage **task scheduling** and **resource allocation** to ensure all timing requirements are consistently met.

Task Activation

- Tasks can be activated based on different criteria:
 - ▶ **Periodic Tasks (Time-driven):** Triggered by regular intervals (e.g., reading a sensor every 10 ms).
 - ▶ **Sporadic Tasks (Event-driven):** Triggered by external occurrences (e.g., an emergency stop button).
- Complex systems integrate both with varying frequencies.

The Super-Loop Model (Polling)

- A simple approach to concurrency is the **super-loop** or **polling** model.
- Tasks are executed sequentially within an infinite loop, with temporal checks to determine when each task should run.

The Super-Loop Model (Polling)

Example: A system with two tasks, Servo_Task (10 ms) and Display_Task (100 ms).

Basic structure with temporal checks

```
while(1) {  
    current_time = get_system_time();  
    if (current_time - t1 >= T1_PERIOD) {  
        run_servo_control(); // 10ms periodicity  
        t1 = t1 + T1_PERIOD; // Schedule next activation  
    }  
    if (current_time - t2 >= T2_PERIOD) {  
        update_display();    // 100ms periodicity  
        t2 = t2 + T2_PERIOD; // Schedule next activation  
    }  
}
```

Cooperative Scheduling

- The super-loop model relies on **cooperative scheduling**.
- Tasks are written as independent functions.
- **Principle:** Each task performs its computation and *yields* the CPU voluntarily upon completion.
- The developer must ensure that no task holds the processor for an excessive duration.
- This model is simple but can lead to issues if a task takes longer than expected, as it can block other tasks from executing on time.

Critical Limitations

- **Blocking and Latency:**

- ▶ If the Display_Task takes 30 ms (e.g., due to slow I2C), the Servo_Task (scheduled for 10 ms) will be delayed.

- **Jitter:**

- ▶ Variations in task start times compromise precise control systems.

- **Energy Management Complexity:**

- ▶ How to determine exactly when the CPU can sleep if the loop is constantly polling time?

- **Scalability:**

- ▶ Adding new tasks requires re-evaluating the temporal impact on all existing tasks.

Facilitating Complex Embedded Systems

- As systems grow in complexity, a higher level of abstraction becomes essential.
- Key Requirements for Scalable Development:
 - ▶ **Pre-emption capability:** The ability to suspend lower-priority tasks immediately to address urgent events.
 - ▶ **Task Isolation:** Decoupling task logic from timing management and scheduling.
 - ▶ **Automatic Power Management:** Efficiently shifting the processor to low-power modes during idle periods without manual intervention.
 - ▶ **Deterministic Scheduling:** Ensuring that timing requirements are strictly met regardless of the number of tasks.

Operating System: An Architectural Solution

An embedded operating system provides the abstraction layer required to satisfy these scalability challenges:

- **Priority-based Pre-emption**
 - ▶ Guaranteed latency for critical tasks.
- **Resource Abstraction**
 - ▶ Decouples task logic from hardware timing.
- **Autonomous Power Control**
 - ▶ Idle-loop power state management.
- **Deterministic Scheduling**
 - ▶ Predictable execution patterns.

Operating System: An Architectural Solution

Core Advantage

By migrating from bare-metal to a platform featuring an operating system, the system gains a **standardised framework** for concurrency, effectively abstracting hardware complexity from application development.

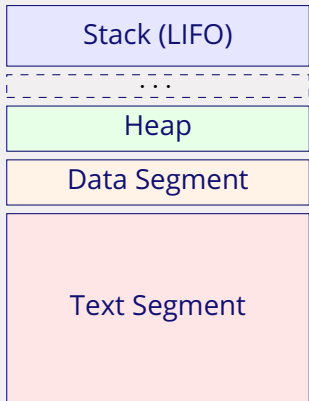
Development within a Managed Environment

The Anatomy of a Program in Execution

When a program runs, it is organised into distinct memory segments, each serving a specific purpose in the execution life cycle.

- **Text Segment:** Contains the executable instructions (read-only).
- **Data Segment:** Holds global and static variables (initialised and uninitialised).
- **Heap:** Manages dynamic memory allocation (grows upward).
- **Stack:** Manages local variables, function parameters, and return addresses (grows downward).

Visualising Memory Layout



Processor State: Execution Pointers

The processor maintains a "snapshot" of the execution using specific registers. These are critical for the operating system to perform context switching.

- **Program Counter (PC) / Instruction Pointer:**
 - ▶ Points to the specific address in the *Text Segment* containing the next instruction to be executed.
- **Stack Pointer (SP):**
 - ▶ Points to the current top of the *Stack*. This tracks the active function frame.
- **Status/Flags Register:**
 - ▶ Holds the current execution state (e.g., ALU flags, interrupt status).

The Dynamic State of Execution

A program's current execution state is a snapshot of its entire environment. To pause or resume a task, the operating system must capture and restore two distinct domains:

Memory State

The data currently residing in RAM:

- **Variables:** Stack and Data segments.
- **Heap:** Dynamically allocated structures.

Processor State

The volatile data held in hardware:

- **Registers:** General purpose and status.
- **Pointers:** PC, SP.

The combination of these two domains defines the task's **context**.

The Classical Multiprogramming Model

In a classical multiprogrammed system, the operating system manages multiple independent applications concurrently, even on a single-core processor.

- **Process-Centric:** Each application is treated as an isolated *process*.
- **Single-Threaded Execution:** Every process contains only one sequential flow of execution (one Program Counter).
- **Pseudo-Parallelism:** Through rapid context switching, the OS gives the illusion that all processes are running simultaneously.

Process Isolation and Context

Since each process is single-threaded, the OS ensures they do not interfere with one another:

- **Memory Protection:** Each process operates within its own private virtual address space.
- **Independent States:** Each process maintains its own stack, heap, and register set.

The Multiprogrammed CPU Lifecycle

The CPU acts as a resource that is continuously reallocated:



Key Mechanism: the Preemption

The OS uses **timer interrupts** to forcibly preempt the running process, ensuring that no single task can monopolise the CPU.

The OS View: Context Switching

The Mechanics of a Context Switch

During a context switch, the operating system must save the current Processor State of the executing process into its *Process Control Block (PCB)* in memory.

It subsequently restores these registers from the new process's PCB, effectively shifting the processor's execution context to a different memory stack and instruction sequence.

The Multi-threaded Model

- Traditional processes are **single-threaded** (one sequence of instructions).
- **Multi-threading** allows a single process to contain multiple concurrent execution flows.
- In embedded systems, these are often referred to as **Tasks** (managed by an RTOS).
- Threads within the same process share the **address space**, making communication and context switching significantly faster than between isolated processes.
 - ▶ The same principle applies to tasks managed by an RTOS.

Shared Resources: The Process Container

All threads/tasks belonging to the same application share specific memory segments:

Memory Segments:

- **Text (Code):** The executable instructions.
- **Data:** Global and static variables.
- **Heap:** Dynamically allocated memory (via `malloc`).

OS Resources:

- Open files/descriptors.
- Signal handlers.

The Private Context: Execution State

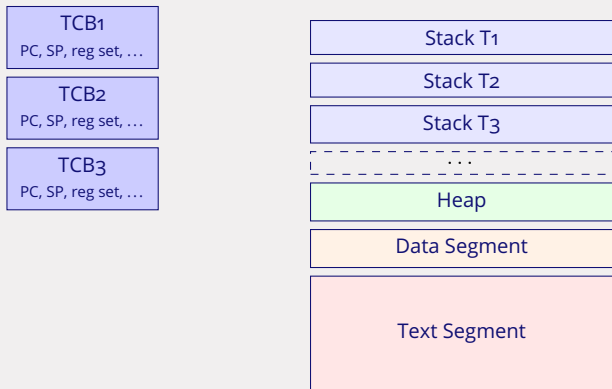
To maintain independence, each thread must have its own **execution context**:

Thread-Specific Resources

- **Program Counter (PC):** Tracks where the thread is in the code.
- **Register Set:** Temporary working values and status flags.
- **Stack Pointer (SP):** Points to a **private stack** for function calls, return addresses, and local variables.

Each thread maintains a **private stack** to ensure memory isolation and prevent the corruption of local data.

Memory Layout in a Multi-threaded Process



Managing Context: The TCB

The OS manages this concurrency through the **Thread Control Block (TCB)**:

- **PCB (Process Control Block)**: Stores process-wide info (memory maps, shared resources).
- **TCB (Thread Control Block)**: Stores the execution state for one specific thread.

Resource	Shared?	Managed via...
Code / Globals	Yes	PCB
Dynamic Heap	Yes	PCB
Registers / PC	No	TCB
Private Stack	No	TCB

Are "Multithread" and "Multitask" the same?

No, although both enable concurrency and share the same core concept: executing multiple code flows by managing a TCB and private execution stacks.

Multithreading (GPOS)

- Context: Typically user-space.
- Memory: Shared virtual address space, via MMU.
- Isolation: Protection between processes; threads are "lightweight" units within a process.

Multitasking (RTOS)

- Context: System-wide tasks.
- Memory: Often a flat memory model (no MMU).
- Isolation: Relies on task design and MPU (if available).

GPOS vs. RTOS: Architectural Differences

Feature	GPOS (e.g., Linux)	RTOS (e.g., FreeRTOS)
Target Hardware	High-performance (MMU required)	Embedded (MMU optional/absent)
Memory Protection	Strict (Virtual Memory)	Often flat (Shared address space)
Scheduling	Throughput/Fairness oriented	Deterministic/Latency oriented
Context Switching	High overhead	Minimal overhead
System Integrity	Isolated via MMU	Relies on software discipline

- **MMU Context:** GPOS platforms use an Memory Management Unit (MMU) for memory protection and virtual memory; RTOS platforms often run in a flat memory model for faster task interaction.
- **Deterministic behaviour:** RTOS prioritises guaranteed response times, whereas GPOS targets overall system throughput.

Memory Models: Virtual vs. Flat

Virtual Memory (GPOS)

- Uses MMU to map virtual addresses to physical RAM.
- Each process has its own isolated virtual address space.
- Enables demand paging and swapping.

Flat Address Space (RTOS)

- No MMU (or disabled); logical equals physical address.
- All tasks share the same address space.
- Faster execution; lower overhead.

Memory Models: Virtual vs. Flat

Key Comparison

	Virtual Memory	Flat Space
Hardware Isolation	MMU required	None/MPU
Complexity	Strong (Hardware)	Weak (Software/MPU)
	High	Low